In this project, I built a home SOC (Security Operations Center) lab using Wazuh, an open-source SIEM platform. The goal was to simulate a real-world security monitoring environment where I can detect and analyze security events.

The setup consists of a Wazuh server running inside Docker containers on WSL (Windows Subsystem for Linux), and a Kali Linux virtual machine acting as a monitored endpoint through the Wazuh agent.

This lab allows me to practice threat detection, log analysis, and security monitoring — core skills for any SOC analyst role.

1. Figure 1 shows the execution of sudo apt update && sudo apt upgrade -y on WSL (Windows Subsystem for Linux). The apt update command refreshes the package list from the repositories, ensuring the system has the latest available software information. The apt upgrade command then installs any pending updates. This step is performed before installing Docker to avoid dependency issues or outdated packages

```
caspian@Caspian:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for caspian:
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3402 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1262 kB]
Fetched 5096 kB in 4s (1146 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  htop libnl-3-200 libnl-genl-3-200
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
caspian@Caspian:~$
```

2. Figure 2 shows the installation of prerequisite packages required for Docker installation. Ca-certificates allows the system to verify SSL connections, curl is used to download files from the internet, gnupg handles encryption keys for verifying Docker's authenticity, and lsb-release provides Linux distribution information needed during setup.

```
caspian@Caspian: ~
caspian@Caspian:~$ sudo apt install -y ca-certificates curl gnupg lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
lsb-release is already the newest version (11.1.0ubuntu4).
ca-certificates is already the newest version (20240203~22.04.1).
curl is already the newest version (7.81.0-1ubuntu1.23).
gnupg is already the newest version (2.2.27-3ubuntu2.5).
The following packages were automatically installed and are no longer required:
  htop libnl-3-200 libnl-genl-3-200
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
caspian@Caspian:~$
```

Figure 3 shows the addition of Docker's official GPG key to the system. A GPG key is a cryptographic signature that verifies the authenticity of software packages. This ensures that the Docker packages we install come from the official Docker source and have not been tampered with.

```
caspian@Caspian:~$ sudo install -m 0755 -d /etc/apt/keyrings
[sudo] password for caspian:
caspian@Caspian:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N) y
caspian@Caspian:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
caspian@Caspian:~$
```

Figure 4 shows the addition of Docker's official repository to the systems's package sources. This tells the package manager where to download Docker from, ensuring we got the latest official version directly from Docker rather than an outdated version from the default Ubuntu repositories.

```
caspian@Caspian:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
://down> https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
ee /etc/> sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Figure 5 shows the installation of Docker engine and its components on WSL. docker-ce is the Docker Community Edition engine, docker-ce-cli is the command line interface, container.io is the container runtime, and docker-buildx-plugin and docker-compose-plugin provide additional functionality for building and orchestrating container.

```
caspian@Caspian:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
containerd.io is already the newest version (2.2.3-1~ubuntu.22.04~jammy).
docker-buildx-plugin is already the newest version (0.33.0-1~ubuntu.22.04~jammy).
docker-ce-cli is already the newest version (5:29.4.0-1~ubuntu.22.04~jammy).
docker-ce is already the newest version (5:29.4.0-1~ubuntu.22.04~jammy).
docker-compose-plugin is already the newest version (5.1.3-1~ubuntu.22.04~jammy).
The following packages were automatically installed and are no longer required:
  htop libnl-3-200 libnl-genl-3-200
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

```
caspian@Caspian:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages [75.1 kB]
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3401 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1262 kB]
Fetched 5171 kB in 4s (1457 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

Figure 6 confirms the successful installation of Docker by displaying the installed version. This verifies that Docker Engine is properly installed and accessible from the command line.

```
caspian@Caspian:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
```

Figure 7 shows the successful execution of the Docker test container hello-world. Docker pulled the test image from Docker Hub and ran it as a container confirming that Docker Engine is properly installed and functioning on WSL. The message ‘Hello from Docker’ confirms that Docker can successfully pull images and run containers.

```
caspian@Caspian:~$ sudo service docker start
 * Starting Docker: docker
caspian@Caspian:~$ sudo usermod -aG docker $USER
caspian@Caspian:~$ newgrp docker
caspian@Caspian:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

caspian@Caspian:~$
```

3. Figure 8 shows the configuration of the vm.max_map_count kernel parameter to 262144. This setting increases the maximum number of memory mapped areas available to processes. This is required by the Wazuh indexer, which is built on OpenSearch and needs a higher memory map limit than the Linux default 65530 to store and index security event data properly.

```
caspian@Caspian:~$ sudo sysctl -w vm.max_map_count=262144
[sudo] password for caspian:
vm.max_map_count = 262144
caspian@Caspian:~$
```

Figure 9 shows the permanent configuration of vm.max_map_count by adding it to /etc/sysctl.conf. This configuration file is loaded every time the system starts, ensuring the setting persists across WSL restarts. The cat command confirms the value successfully written to the file.

```
caspian@Caspian:~$ echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
vm.max_map_count=262144
caspian@Caspian:~$ cat /etc/sysctl.conf | grep max_map_count
vm.max_map_count=262144
vm.max_map_count=262144
```

Figure 10 shows the cloning of the official Wazuh Docker repository from GitHub. The -b v4.14.4 flag specifies version 4.14.4 for stability, while –depth=1 downloads only the latest snapshot of the repository rather than its full history, reducing download size and time. This repository contains all the Docker Compose configuration files needed to deploy the Wazuh stack.

```
caspian@Caspian:~$ git clone https://github.com/wazuh/wazuh-docker.git -b v4.14.4 --depth=1
Cloning into 'wazuh-docker'...
remote: Enumerating objects: 155, done.
remote: Counting objects: 100% (155/155), done.
remote: Compressing objects: 100% (126/126), done.
remote: Total 155 (delta 25), reused 80 (delta 17), pack-reused 0 (from 0)
Receiving objects: 100% (155/155), 94.13 KiB | 1.59 MiB/s, done.
Resolving deltas: 100% (25/25), done.
Note: switching to '7af31ddfb4d7dd72acbd0789728185c525a64755'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

caspian@Caspian:~$
```

Figure 11 shows the contents of the single-node directory inside the cloned Wazuh Docker repository. This directory contains the docker-compose.yml file which defines all three Wazuh components (manager, indexer, and dashboard) as separate containers, along with certificate generation and configuration files needed for deployment.

```
caspian@Caspian:~$ cd wazuh-docker/single-node
caspian@Caspian:~/wazuh-docker/single-node$ ls
README.md  config  docker-compose.yml  generate-indexer-certs.yml
caspian@Caspian:~/wazuh-docker/single-node$
```

4. Figure 12 shows the generation of SSL certificates for the Wazuh deployment using the official Wazuh certificate generator Docker image. SSL certificates enable encrypted and authenticated communication between the Wazuh manager, indexer and dashboard components.The –rm flag ensures the generator container is automatically removed after completing its tasks.

```
caspian@Caspian:~/wazuh-docker/single-node$ docker compose -f generate-indexer-certs.yml run --rm generator
[+]  1/1te 1/1
 ⠿ Network single-node_default Created
Container single-node-generator-run-d305bd039cba Creating
Container single-node-generator-run-d305bd039cba Created
Checking https://packages.wazuh.com/4.14/wazuh-certs-tool.sh ...
Downloaded wazuh-certs-tool.sh from https://packages.wazuh.com/4.14/
19/04/2026 11:04:01 INFO: Verbose logging redirected to //wazuh-certificates-tool.log
19/04/2026 11:04:01 INFO: Generating the root certificate.
19/04/2026 11:04:01 INFO: Generating Admin certificates.
19/04/2026 11:04:01 INFO: Admin certificates created.
19/04/2026 11:04:01 INFO: Generating Wazuh indexer certificates.
19/04/2026 11:04:02 INFO: Wazuh indexer certificates created.
19/04/2026 11:04:02 INFO: Generating Filebeat certificates.
19/04/2026 11:04:02 INFO: Wazuh Filebeat certificates created.
19/04/2026 11:04:02 INFO: Generating Wazuh dashboard certificates.
19/04/2026 11:04:02 INFO: Wazuh dashboard certificates created.
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker
caspian@Caspian:~/wazuh-docker/single-node$
```

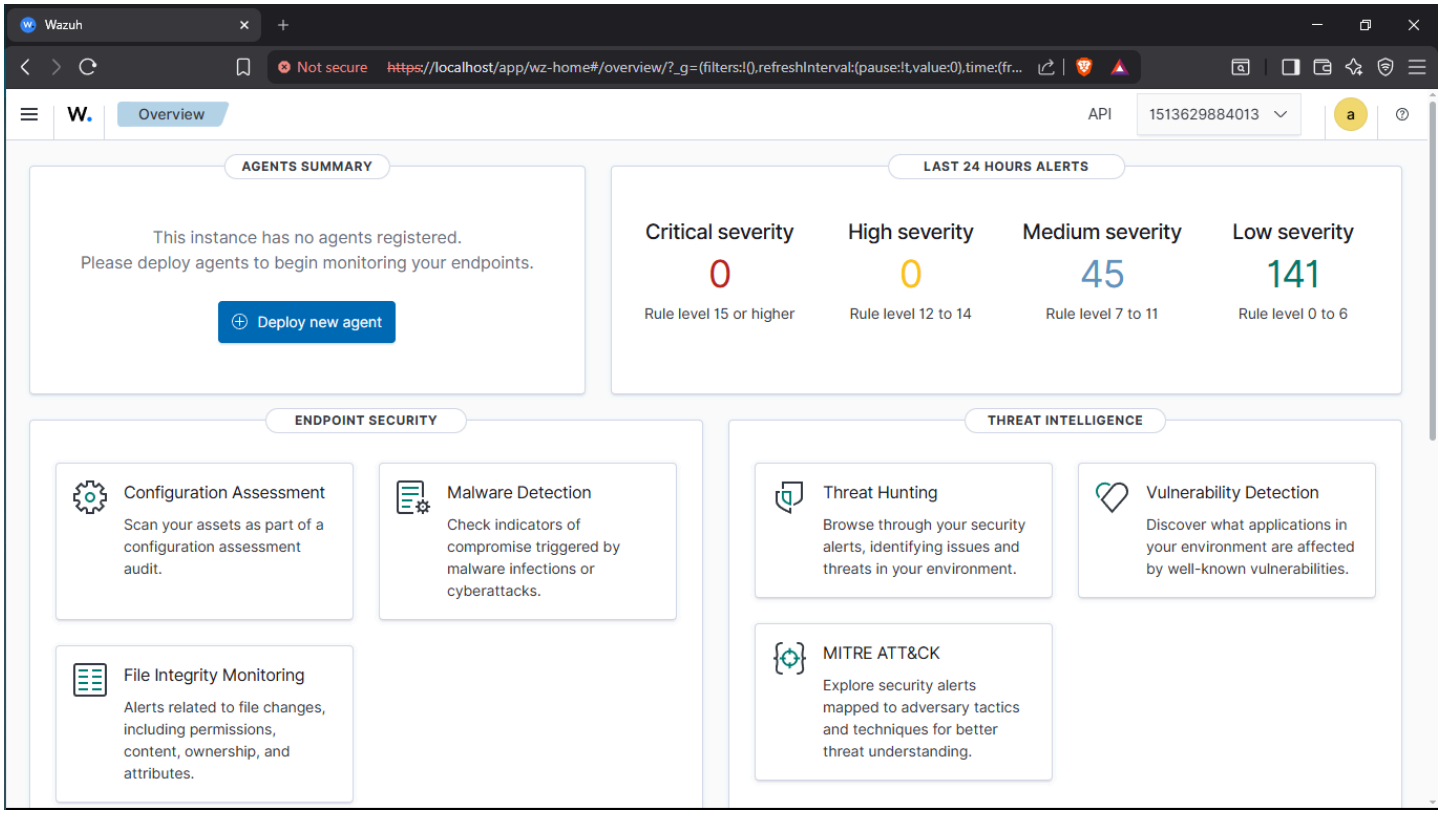
Figure 13 shows the deployment of the Wazuh stack using Docker Compose. The -d flag runs the container in detached mode, allowing them to run in the background. Docker Compose reads the docker-compose.yml file and simultaneously starts three containers: the Wazuh manager, Wazuh indexer, and Wazuh dashboard, each with their own isolated environment and persistent storage volumes.

```
caspian@Caspian:~/wazuh-docker/single-node$ docker compose up -d
[+] up 17/17
 ✔ Volume single-node_wazuh_integrations      Created                                                    0.0s
 ✔ Volume single-node_wazuh_etc               Created                                                    0.0s
 ✔ Volume single-node_wazuh_agentless         Created                                                    0.0s
 ✔ Volume single-node_filebeat_etc            Created                                                    0.0s
 ✔ Volume single-node_wazuh-dashboard-config  Created                                                    0.0s
 ✔ Volume single-node_wazuh-dashboard-custom  Created                                                    0.0s
 ✔ Volume single-node_wazuh_var_multigroups   Created                                                    0.0s
 ✔ Volume single-node_filebeat_var            Created                                                    0.0s
 ✔ Volume single-node_wazuh_logs              Created                                                    0.0s
 ✔ Volume single-node_wazuh_queue             Created                                                    0.0s
 ✔ Volume single-node_wazuh_active_response   Created                                                    0.0s
 ✔ Volume single-node_wazuh_wodles            Created                                                    0.0s
 ✔ Volume single-node_wazuh-indexer-data      Created                                                    0.0s
 ✔ Volume single-node_wazuh_api_configuration Created                                                    0.0s
 ✔ Container single-node-wazuh.indexer-1      Started                                                    0.7s
 ✔ Container single-node-wazuh.manager-1      Started                                                    0.9s
 ✔ Container single-node-wazuh.dashboard-1    Started                                                    0.9s
caspian@Caspian:~/wazuh-docker/single-node$
```

Figure 14 shows the status of the Wazuh containers after deployment using docker compose ps. All three containers – the Wazih manager, indexer, and dashboard – are shown with a running status, confirming that the Wazuh stack has been successfully deployed. The dashboard is accessible on port 443, which is the standard HTTPS port.

```
caspian@Caspian:~/wazuh-docker/single-node$ docker compose ps
NAME                          IMAGE                        COMMAND                  SERVICE          CREATED        STATUS         PORTS
single-node-wazuh.dashboard-1   wazuh/wazuh-dashboard:4.14.4   "/entrypoint.sh"         wazuh.dashboard   3 minutes ago   Up 3 minutes   443/tcp, 0.0.0.0:443->5601/tcp, [::]:443-
>5601/tcp
single-node-wazuh.indexer-1     wazuh/wazuh-indexer:4.14.4     "/entrypoint.sh open…"   wazuh.indexer     3 minutes ago   Up 3 minutes   0.0.0.0:9200->9200/tcp, [::]:9200->9200/t
cp
single-node-wazuh.manager-1     wazuh/wazuh-manager:4.14.4     "/init"                  wazuh.manager     3 minutes ago   Up 3 minutes   0.0.0.0:1514-1515->1514-1515/tcp, [::]:15
14-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp
```

Figure 15 Shows the Wazuh dashboard after successful authentication. The dashboard provides a centralized view of all security events, agent status, alerts, and compliance information collected across monitored endpoints.



5. Figure 16 shows the IP of the WSL network interface retrieved using the ip addr show command. This IP address is the address of the Wazuh manager that Kali Linux will use to communicate with the Wazuh server.

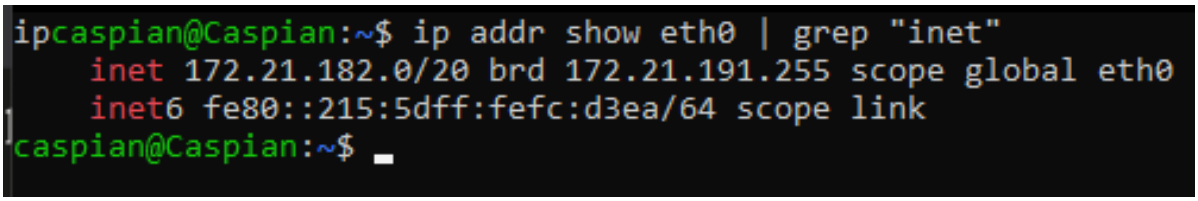


Figure 17 shows the addition of the official Wazuh repository to the Kali Linux package manager. The GPG key is first imported to verify the authenticity of Wazuh packages,and the repository URL is then added to the system's package sources. This ensures the Wazuh agent is downloaded from the official and trusted Wazuh source.

```
Session  Actions  Edit  View  Help
┌──(caspian0㉿Caspian)-[~]
└─$ curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --no-default-keyring \
--keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import && \
sudo chmod 644 /usr/share/keyrings/wazuh.gpg
[sudo] password for caspian0:
gpg: key 96B3EE5F29111145: "Wazuh.com (Wazuh Signing Key) <support@wazuh.com>" not changed
gpg: Total number processed: 1
gpg:              unchanged: 1

┌──(caspian0㉿Caspian)-[~]
└─$ echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main
" | \
sudo tee /etc/apt/sources.list.d/wazuh.list
deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main

┌──(caspian0㉿Caspian)-[~]
└─$
```

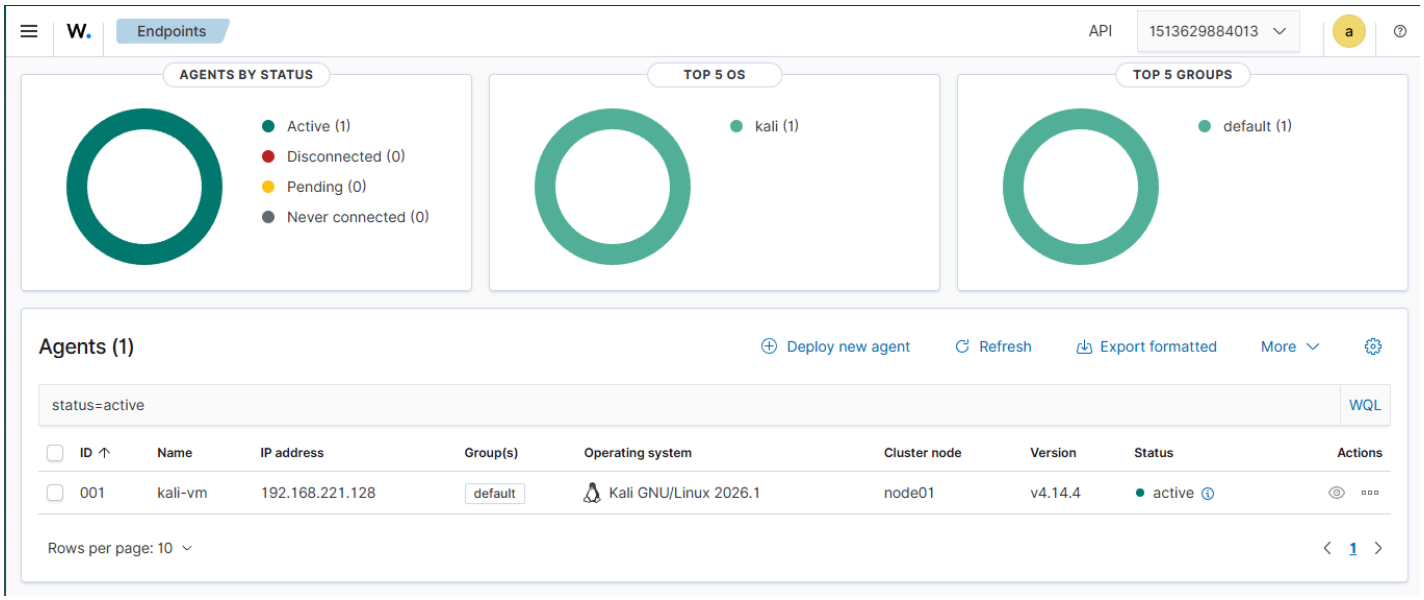
Figure 18 shows the installation of the Wazuh agent on Kali Linux. The Wazuh_Manager environment variable specifies the IP address of the Wazuh manager so the agent knows where to send security data. The WAZUH_AGENT_NAME variable assigns the name kali-vm to this agent making it identifiable in the Wazuh dashboard.

```
Session   Actions   Edit   View   Help
└─$ sudo WAZUH_MANAGER="172.21.182.0" \
WAZUH_AGENT_NAME="kali-vm" \
apt install -y wazuh-agent
The following packages were automatically installed and are no longer required:
  curlftpfs                 libfuse2t64              libpostproc58        python3-aiocache
  libann0                   libgav1-1                librubberband2       python3-aiomcache
  libaudio2                 libgdk-pixbuf2.0-bin     libsframe2           python3-fs
  libavfilter10             libicu76                 libsimdutf27         python3-pysnmp4
  libavformat61             liblab-gamut1            libsnmp40t64         python3-terminaltables
  libblosc2-4               libmjpegutils-2.1-0t64   libsphinxbase3t64    python3-wapiti-arsenic
  libcdt5                   libmpeg2encpp-2.1-0t64   libswscale8          python3-yaswfp
  libcgraph6                libmplex2-2.1-0t64       libtsk19t64          ruby-unf-ext
  libconfig-inifiles-perl   libmupdf25.1             libvdpau-va-gl1      vdpau-driver-all
  libcrypt-dev              libplacebo351            libzxing3
  libdlt2                   libpocketsphinx3         pocketsphinx-en-us
Use 'sudo apt autoremove' to remove them.

Installing:
  wazuh-agent

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 303
  Download size: 13.2 MB
  Space needed: 48.9 MB / 29.5 GB available

Get:1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.14.4-1 [13.2 MB]
Fetched 13.2 MB in 1s (18.5 MB/s)
Preconfiguring packages ...
Selecting previously unselected package wazuh-agent.
(Reading database… 439051 files and directories currently installed.)
Preparing to unpack …/wazuh-agent_4.14.4-1_amd64.deb…
Unpacking wazuh-agent (4.14.4-1)…
Setting up wazuh-agent (4.14.4-1)…
```

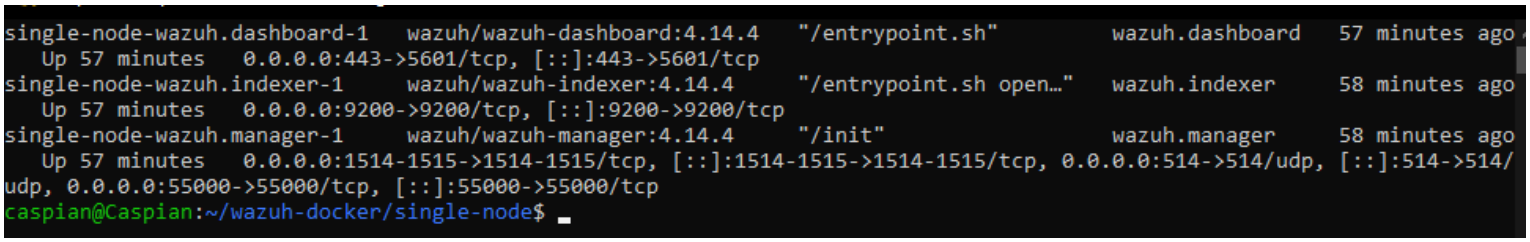
Figure 19 shows the status of the Wazuh agent service on Kali Linux after installation. The active (running) status confirms the agent is successfully running. The listed processes – wazuh-agentd, wazuh-execd, wazuh-syscheckd, wazuh-logcollector, and wazuh-modulesd – represent the individual components of the agent responsible for communication, execution, file integrity monitoring, log collection, and security module management respectively.

```
Session   Actions   Edit   View   Help

┌──(caspian0㉿Caspian)-[~]
└─$ sudo systemctl enable wazuh-agent
[sudo] password for caspian0:
Created symlink '/etc/systemd/system/multi-user.target.wants/wazuh-agent.service' → '/usr/lib/systemd/system/waz
uh-agent.service'.

┌──(caspian0㉿Caspian)-[~]
└─$ sudo systemctl start wazuh-agent

┌──(caspian0㉿Caspian)-[~]
└─$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
     Active: active (running) since Sun 2026-04-19 22:36:46 PDT; 23s ago
 Invocation: 193c6a75afd24c2c9cc86a84780af8a0
    Process: 33323 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Process: 35108 ExecReload=/usr/bin/env /var/ossec/bin/wazuh-control reload (code=exited, status=0/SUCCESS)
      Tasks: 34 (limit: 9244)
     Memory: 186.8M (peak: 187.9M)
        CPU: 19.698s
     CGroup: /system.slice/wazuh-agent.service
             ├─33372 /var/ossec/bin/wazuh-agentd
             ├─35226 /var/ossec/bin/wazuh-execd
             ├─35253 /var/ossec/bin/wazuh-syscheckd
             ├─35263 /var/ossec/bin/wazuh-logcollector
             ├─35296 /var/ossec/bin/wazuh-modulesd
             ├─35588 sh -c -- "/bin/ps -p 15 2> /dev/null"
             └─35590 /bin/ps -p 15

Apr 19 22:37:01 Caspian env[35108]: Killing wazuh-execd ...
Apr 19 22:37:05 Caspian env[35108]: Wazuh v4.14.4 Stopped
Apr 19 22:37:06 Caspian env[35108]: Starting Wazuh v4.14.4 ...
Apr 19 22:37:07 Caspian env[35108]: Started wazuh-execd ...
Apr 19 22:37:07 Caspian env[35108]: wazuh-agentd already running ...
Apr 19 22:37:07 Caspian env[35108]: Started wazuh-syscheckd ...
Apr 19 22:37:08 Caspian env[35108]: Started wazuh-logcollector ...
Apr 19 22:37:08 Caspian env[35108]: Started wazuh-modulesd ...
Apr 19 22:37:10 Caspian env[35108]: Completed.
Apr 19 22:37:10 Caspian systemd[1]: Reloaded wazuh-agent.service - Wazuh agent.

┌──(caspian0㉿Caspian)-[~]
└─$
```

Figure 20 shows the Kali Linux agent successfully enrolled and visible in the Wazuh dashboard under Endpoints section. The agent kali-vm is shown with an Active status and IP address 192.168.221.128, confirming that it has successfully connected to the Wazuh manager and is actively sending security telemetry data such as logs, file integrity events, and system activity for monitoring and analysis.

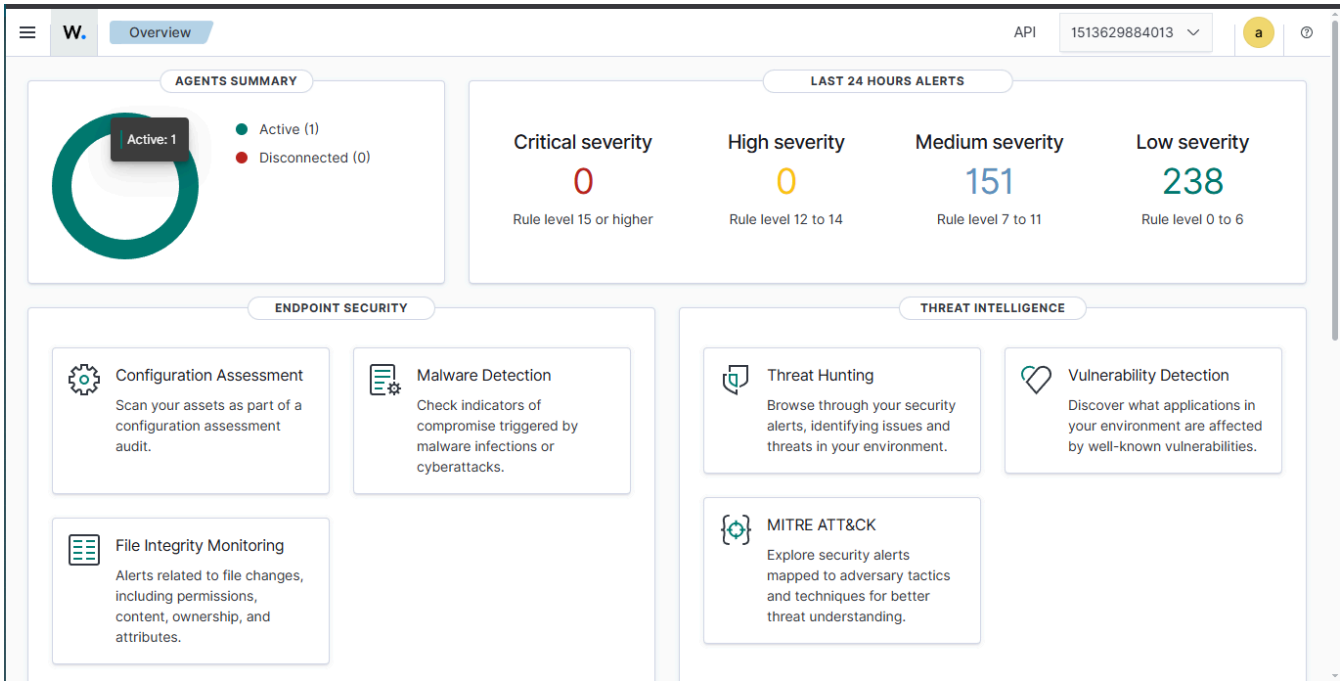


6. Figure 21shows all three Wazuh containers running successfully via docker compose ps. The Wazuh manager, indexer, and dashboard containers are all in a running state, confirming the Wazuh server stack is fully operational.



```
single-node-wazuh.dashboard-1   wazuh/wazuh-dashboard:4.14.4   "/entrypoint.sh"         wazuh.dashboard   57 minutes ago
   Up 57 minutes   0.0.0.0:443->5601/tcp, [::]:443->5601/tcp
single-node-wazuh.indexer-1     wazuh/wazuh-indexer:4.14.4     "/entrypoint.sh open…"   wazuh.indexer     58 minutes ago
   Up 57 minutes   0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
single-node-wazuh.manager-1     wazuh/wazuh-manager:4.14.4     "/init"                  wazuh.manager     58 minutes ago
   Up 57 minutes   0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/
udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp
caspian@Caspian:~/wazuh-docker/single-node$
```

Figure 22 shows the Wazuh dashboard overview page. This page provides a high level summary of all security events, active agents, and recent alerts across all monitored endpoints.



Conclusion

In this project, I successfully deployed a home SOC(Security Operations Center) lab using Wazuh, and open-source SIEM (Security Information and Event Management) platform. The Wazuh server was deployed using Docker on WSL (Windows Subsystem for Linux) since Wazuh is not natively compatible with Windows. Docker allowed all three Wazuh components to run efficiently inside WSL containers.

The three core components of the Wazuh server each serve a specific purpose. The Wazuh Indexer acts as a full text search and analytics engine that stores and indexes all collected security data. The Wazuh Manager serves as the brain of the operation, receiving data from agents, applying detection rules, and handling integrity monitoring and active responses. The Wazuh Dashboard provides the web user interface where SOC analysts can monitor alerts, investigate security events, and manage agents.

Before deploying Wazuh, the vm.max_map_count kernel parameter was configured to 262144. This was necessary because the Wazuh indexer requires a high number of memory mapped areas than the Linux default to 65530, and without this configuration the indexer would fail to start.

A Wazuh agent was successfully installed and enrolled on a Kali Linux virtual machine running on VMware. The agent automatically starts when Kali boots up and continuously sends security telemetry–including logs, file integrity events, and system activity–to the Wazuh manager for analysis.

This lab simulates a real world SOC environment and serves as a foundation for further security projects including threat detection, incident response, and log analysis. The skills practiced in this project include:

- Deploying containerized using Docker
- Configuring Linux kernel parameters for security tools
- Setting up a SIEM platform for security monitoring
- Enrolling and managing Wazuh agents
- Navigating a SOC dashboard for threat detection and analysis